

	Computación	Docente: Vladimir Robles Bykbaev
	VISIÓN ARTIFICIAL	Período Lectivo: Abril – Agosto 2026

		FORMATO DE GUÍA DE PROYECTO INTEGRADOR	
CARRERA: COMPUTACIÓN		ASIGNATURA: VISIÓN ARTIFICIAL	
NRO. PRÁCTICA:	2-2	TÍTULO DEL PROYECTO: Proyecto Integrador Final - Desarrollo de Sistema Inteligente de Visión Estéreo y Detección de Vehículos Específicos en Entornos Reales.	
OBJETIVO: Reforzar los conocimientos adquiridos en clase sobre el trabajo con las etapas de preprocesamiento de imágenes y extracción de características locales y globales y el empleo de técnicas del dominio de la frecuencia y de aprendizaje de máquina a fin de realizar tareas de reconocimiento de patrones.			
INSTRUCCIONES:		1. Revisar el contenido teórico del tema, las presentaciones, tutoriales y video-lecturas disponibles en el ambiente virtual de aprendizaje.	
		2. Profundizar los conocimientos revisando los libros guías, los enlaces contenidos en los objetos de aprendizaje y la documentación disponible en fuentes académicas en línea	
		3. Si emplea código de terceros debe citar y en caso de emplear herramientas generadoras de código (LLMs) debe entender la lógica y el funcionamiento de las líneas de código empleadas (citándolas).	
		4. Deberá generar un informe empleando una herramienta Web 2.0 o similar y un video-blog en inglés explicando los principales aspectos de la propuesta planteada.	
ASIGNATURAS Y CONOCIMIENTOS RELACIONADOS AL PROYECTO INTEGRADOR			
Asignatura	Contenido de la materia	Contenido al que aporta de la asignatura de Visión por Computador o requerimientos del Proyecto	Porcentaje de aporte al proyecto
Álgebra Lineal	Matrices (operaciones con matrices)	Operaciones sobre grupos de puntos de la imagen (convolución, filtro de la media, filtro de la mediana, filtro Gaussiano). Reducción de ruido. Operaciones morfológicas. HOG.	25%
Cálculo Diferencial	La derivada	Detección de Bordes (Operadores Básicos, Prewitt, Sobel, Laplaciano, Canny)	20%
Estadística	Introducción a la estadística y conjuntos Probabilidad Variables aleatorias y distribuciones de probabilidad	Ecualización de Histograma	25%
Programación y plataformas Web	Desarrollo de aplicaciones Web		15%
Inteligencia Artificial	Redes Neuronales	Aplicación de visión por computador con aprendizaje profundo.	15%
Visión por Computador	Espacios de Color Operaciones sobre los puntos de la imagen Operaciones Morfológicas Detección de Bordes Histogram Oriented Gradients (HOG) Cascadas de Haar	N/A	N/A

PROBLEMA QUE BUSCA SOLUCIONAR EL PROYECTO INTEGRADOR

En el contexto actual de la visión artificial aplicada a la robótica y el monitoreo urbano, existe una creciente necesidad de sistemas que integren hardware de bajo costo con algoritmos robustos. Este proyecto integrador busca abordar dos desafíos técnicos reales: la estimación de profundidad mediante visión estereoscópica lidiando con latencia y desincronización de hardware, y la clasificación de objetos específicos (vehículos ecuatorianos) en condiciones variables de entorno.

Al combinar herramientas como OpenCV (C++ y Python) con microcontroladores (ESP32), los estudiantes consolidarán sus conocimientos teóricos en un entorno práctico, demostrando la capacidad de construir soluciones que van desde el control de firmware y preprocesamiento básico hasta la inferencia clásica de machine learning.

PARTE I. INTERCICLO: ESTIMACIÓN DE PROFUNDIDAD ESTÉREO Y REALIDAD AUMENTADA

Al realizar un proyecto integrador que involucre visión estereoscópica, los estudiantes deben emplear dos cámaras de bajo costo (modelos ESP32-CAM-MB o ESP32-XIAO-SP) para calcular la distancia de un objeto posicionado en el centro del campo de visión.

Este proceso exige un control riguroso desde el hardware hasta el software. Los estudiantes deberán realizando pruebas de modificación en el firmware de las cámaras para deshabilitar controles automáticos (AEC, AGC, AWB) y ajustar frecuencias de reloj para garantizar la paridad lumínica y reducir la latencia de red. En el software, deberán implementar algoritmos de calibración estéreo, rectificación de imágenes, y cálculo de mapas de disparidad (SGBM o similares), aplicando técnicas de preprocesamiento adaptativo (como CLAHE o similares) y filtros de posprocesamiento (como WLS o similares) para obtener lecturas estables en milímetros o centímetros.

Finalmente, sobre la imagen rectificada en tiempo real, cada grupo deberá superponer un efecto visual dinámico de realidad aumentada (ej. un aura de energía, fuego, distorsión, etc.) cuyo tamaño o comportamiento reaccione a la distancia calculada. **Restricción estricta: Cada grupo debe diseñar su propio efecto visual y establecer sus propios parámetros algorítmicos. No se aceptarán dos grupos con el mismo efecto.**

Al realizar la Parte I del proyecto integrador, centrada en la estimación de profundidad estéreo y el desarrollo de efectos de realidad aumentada, los estudiantes consolidan de manera práctica los conocimientos de las UNIDADES 1 y 2. En primer lugar, la configuración y calibración del arreglo estéreo con las ESP32-CAM requiere una comprensión profunda del modelo Pinhole, los elementos de formación de imágenes y el manejo del ruido originado en la digitalización, aspectos cubiertos en la UNIDAD 1 (Formación y adquisición de imágenes). Posteriormente, para lograr mapas de disparidad estables y lecturas precisas de distancia en centímetros, los estudiantes deben implementar técnicas de la UNIDAD 2 (Operaciones y preprocesamiento de imágenes). Esto incluye la aplicación obligatoria de algoritmos para la reducción de ruido (como filtros CLAHE o WLS) y posibles operaciones morfológicas, con el objetivo de suavizar y realzar los bordes de los objetos detectados.

Estas actividades permiten a los alumnos alcanzar los **Resultados de Aprendizaje: 1 (Identifica características, espacios de color y elementos de formación de imágenes) y 2 (Desarrolla algoritmos para preprocesamiento y manipulación de imágenes)**, demostrando su capacidad para comprender los fenómenos de captación y transformar digitalmente dichas imágenes para extraer información útil en un sistema de visión artificial aplicado.

PARTE II. FINAL: DETECCIÓN DE VEHÍCULOS DEL ENTORNO ECUATORIANO MEDIANTE HOG

En la segunda parte del ciclo, el desafío de detectar y clasificar vehículos específicos del entorno ecuatoriano mediante Histogramas de Gradientes Orientados (HOG) permite abordar directamente los contenidos de las UNIDADES 3 y 4. Para entrenar clasificadores robustos (como SVM o Cascadas de Haar), los estudiantes deben procesar su dataset personalizado empleando técnicas de la UNIDAD 3 (Extracción de características), aplicando algoritmos como Histogram of Oriented Gradients (HOG) o Local Binary Patterns (LBP) para describir la forma y textura de los vehículos. Seguidamente, la integración de estos descriptores con el modelo de clasificación y la inferencia en tiempo real cubren los objetivos de la UNIDAD 4 (Fundamentos de descriptores avanzados y reconocimiento de patrones), introduciéndolos también a los conceptos de aprendizaje de máquina. De esta forma, se da cumplimiento a los **Resultados de Aprendizaje: 3 y 4**, capacitando a los estudiantes en el desarrollo de algoritmos para la extracción de características y en la implementación de aplicaciones orientadas al reconocimiento de patrones para resolver problemas del entorno.

Cada grupo deberá construir, curar y documentar su propio conjunto de datos (dataset) de al menos 4.000

	Computación	Docente: Vladimir Robles Bykbaev
	VISIÓN ARTIFICIAL	Período Lectivo: Abril – Agosto 2026

imágenes positivas y 4.000 negativas centrado en un vehículo objetivo específico del ecosistema de transporte del Ecuador (ej., modelo particular de bus, taxi, o transporte de carga). No pueden haber dos o más grupos que entrenen sus modelos de reconocimiento de patrones sobre el mismo tipo de vehículo o utilicen el mismo dataset base. El incumplimiento de esta directriz anulará la evaluación de la actividad

Este proyecto se enmarca dentro de un contexto práctico y profesional donde los estudiantes deben aplicar técnicas de visión por computador para el monitoreo inteligente, integrando conceptos de extracción de características locales y globales. Los estudiantes se enfrentarán al reto de la escasez de datos prefabricados, debiendo construir, curar y aumentar su propio dataset para entrenar modelos resilientes a las condiciones viales reales

ACTIVIDADES POR DESARROLLAR

- **Desarrollar una aplicación basada en visión artificial y organizada en dos etapas. La primera etapa se debe presentar en el interciclo, y la segunda etapa previo a finalizar el ciclo lectivo. La aplicación deberá cumplir con las siguientes especificaciones:**
 - a) **Parte I (Interciclo):**
 - Debe desarrollar una aplicación de escritorio usando OpenCV C++ que implemente un sistema de visión estereoscópica en tiempo real para el cálculo de distancias y la generación de efectos dinámicos de Realidad Aumentada (AR). Para ello, deberá configurar y sincronizar un arreglo de dos microcontroladores con cámara, específicamente los modelos ESP32-CAM-MB o ESP32-XIAO-SP.
 - Con esta arquitectura de hardware, deberá capturar los flujos de video, realizar la calibración y rectificación estéreo, y generar un mapa de disparidad denso para calcular la distancia en milímetros o centímetros del objeto ubicado en el centro de la imagen.
 - Para garantizar la viabilidad y estabilidad del sistema en tiempo real, su diseño algorítmico se sugiere incluir los siguientes parámetros y técnicas:
 - **Optimización de Firmware (Cámaras):** Modificar el código fuente de los microcontroladores para realizar pruebas al activar o desactivar los controles automáticos de los sensores (Auto Exposición - AEC, Auto Ganancia - AGC, y Balance de Blancos - AWB) garantizando paridad lumínica. Asimismo, se sugiere probar a reducir la frecuencia del reloj externo (XCLK, por ejemplo a 6 MHz) para priorizar la integridad de los paquetes de video y reducir el jitter de red.
 - **Preprocesamiento y Filtros (OpenCV):** Probar aplicando ecualización adaptativa de histograma (CLAHE) o técnicas similares a las imágenes rectificadas antes de procesarlas con el algoritmo SGBM (Semi-Global Block Matching) para resaltar texturas.
 - **Posprocesamiento y Estabilización:** Filtrar el mapa de disparidad crudo utilizando el filtro WLS (Weighted Least Squares) para rellenar huecos y suavizar bordes. La lectura final de la distancia Z debe ser estabilizada algorítmicamente (ej. Filtro de Kalman 1D o Media Móvil) para eliminar las fluctuaciones subpíxel.
 - **Efecto Visual AR (Restricción Estricta):** Sobre la imagen a color rectificada, el sistema deberá dibujar e integrar un efecto visual computacional (por ejemplo, un aura estilo "Super Saiyajin", fuego, distorsión, escudo de energía, etc.) posicionado sobre el objeto central. El tamaño o comportamiento de **este efecto debe reaccionar en tiempo real a la distancia calculada**. Cada grupo debe diseñar su propio efecto gráfico y definir sus propios parámetros de SGBM. **Bajo ninguna circunstancia se aceptarán dos grupos con el mismo efecto o diseño visual.**
 - Un ejemplo del proceso a seguir se observa en la Ilustración 1. **Tome en cuenta que esta es la línea base y deberá diseñar su propio proceso, indicando qué técnicas aplicó realizar las diferentes etapas requeridas (toma de fotografías, calibración estereoscópica, cálculo de distancia, etc.):**

Proyecto Integrador Visión por Computador - Parte I (Visión Estéreo y Realidad Aumentada)

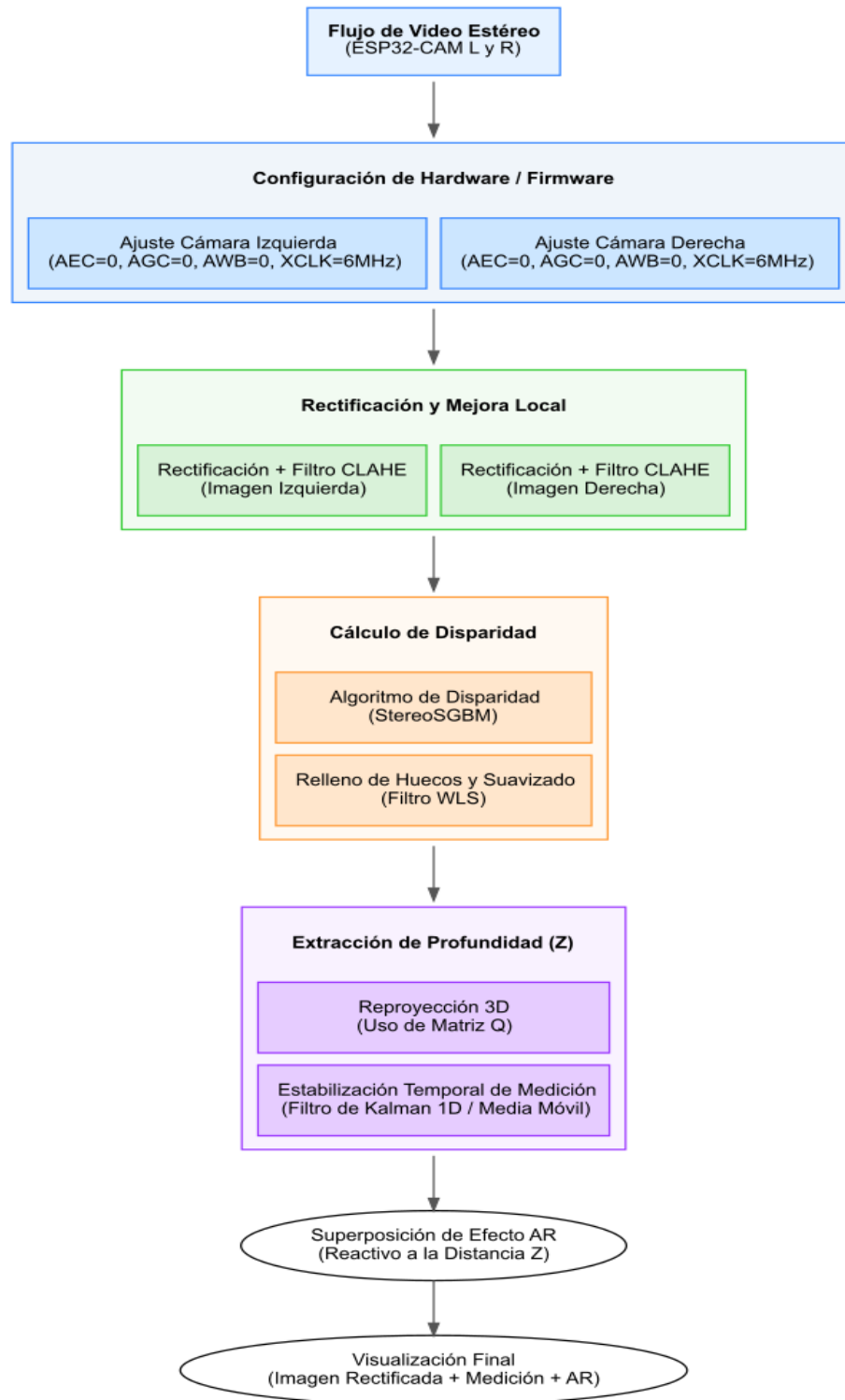


Ilustración 1: Proceso ejemplo seguido en la solución del problema planteado en la primera parte del proyecto integrador

- Un ejemplo del resultado a obtener (mapa de disparidad procesado y cálculo de la distancia sobre la imagen rectificadas) se observa en las ilustraciones contenidas en la Tabla 1. **Tome en cuenta que esta es la línea base y deberá diseñar su propio proceso, indicando y justificando qué técnicas aplicó para estabilizar la medición.**

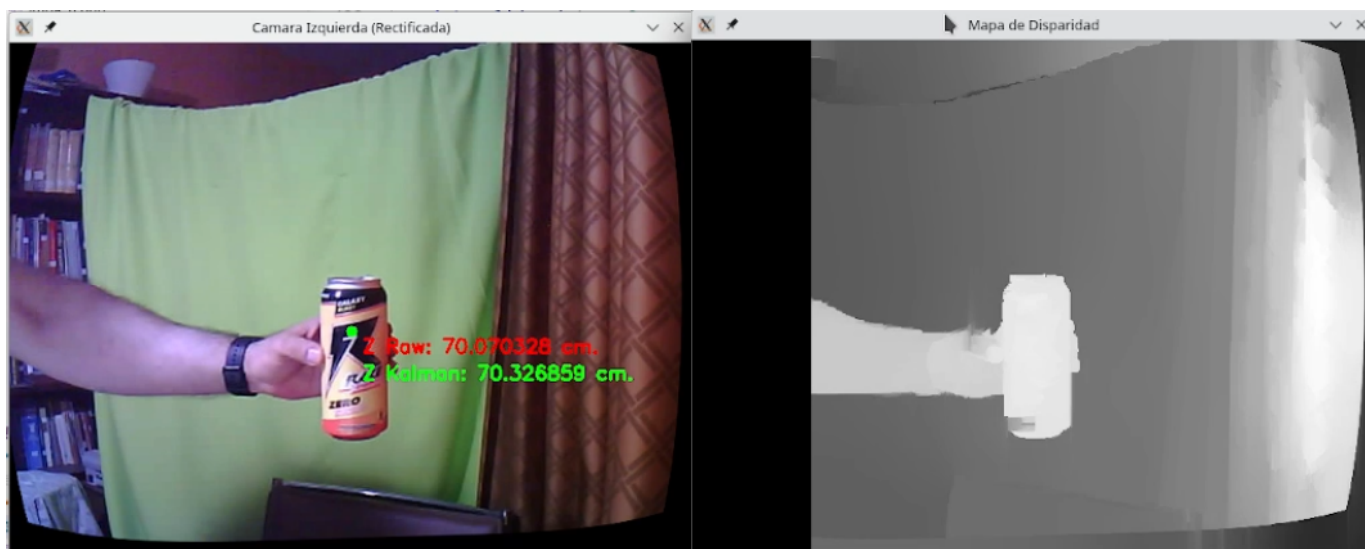
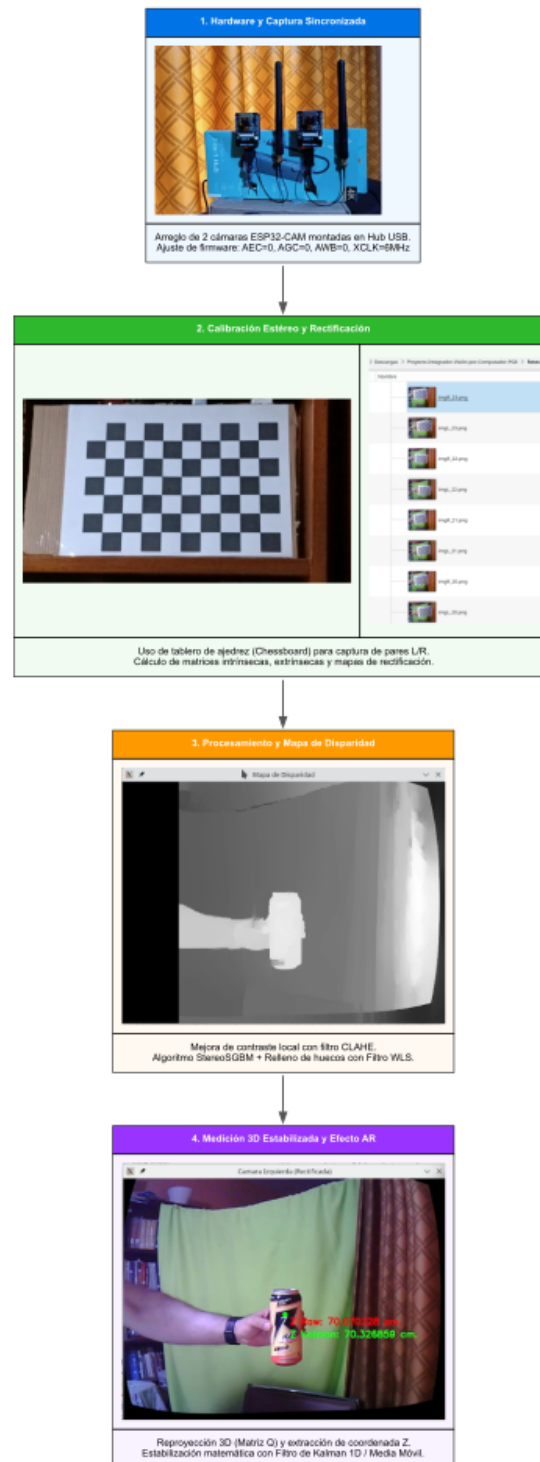


Imagen rectificadas resultado de la captura de las dos cámaras. Se aprecia la distancia y la distancia usando el Filtro de Kalman.

Mapa de disparidad de la imagen. Se observa como el mapa es muy nítido: los bordes de la lata y del brazo están perfectamente definidos, y la separación del fondo es casi perfecta.

Tabla 1: Ejemplo de dos imágenes extraídas obtenidas al realizar el proceso completo de visión estereoscópica.

- Los aspectos y funcionalidades con los que debe contar la aplicación de escritorio desarrollada para la Parte I son los que se indican seguidamente (Ilustración 2):
 - La aplicación de escritorio permitirá procesar el flujo de video sincronizado y en tiempo real proveniente de las dos cámaras (ESP32-CAM), y generará la imagen principal con el efecto dinámico de Realidad Aumentada (AR) superpuesto, permitiendo visualizar a la par las operaciones intermedias realizadas (imágenes individuales rectificadas y el mapa de disparidad).
 - La aplicación debe contar con parámetros estructurados o controles para realizar el ajuste fino del algoritmo StereoSGBM o similar, configurar los filtros de mitigación de ruido (CLAHE, WLS o similar) y calibrar la estabilización de la medición, en función de lo que haya diseñado y establecido cada grupo para su efecto visual.
 - Los archivos de configuración esenciales (como las matrices de calibración estéreo en formato **YAML**) y las capturas de evidencia de las fases intermedias y finales deberán almacenarse y gestionarse correctamente en una carpeta del proyecto.
 - Las operaciones implementadas para rectificar las imágenes, calcular el mapa de disparidad y extraer la coordenada de profundidad (**Z**) deberán estar sustentadas en un proceso lógico y matemático robusto, que garantice una medición precisa y estable capaz de mitigar las fluctuaciones de iluminación del entorno real.



Flujo de Trabajo: Visión Estéreo con ESP32-CAM y AR

Ilustración 2: Ejemplo que ilustra el proceso a seguir para calibrar la cámara, obtener el mapa de disparidad y calcular la distancia en un objeto en el centro de la imagen.

- Debe emplear algunas de las siguientes técnicas de visión artificial que permitan mejorar el pre-procesamiento de la imagen (especialmente para atenuar ruido producido por la iluminación):

- Thersholding
- Contrast Stretching 100%
- Binarización por umbral de color
- Operaciones sobre los puntos de la imagen (NOT, AND, OR, XOR)
- Detección de Bordes
- Manipulación de pixeles.
- Filtros de suavizado.
- Operaciones morfológicas
- **Otras técnicas que se investigue**
- Aspectos generales a tomar en consideración:
 - Se debe diseñar la propuesta del efecto propia y única, donde la gráfica del efecto debe realizarse usando manipulación de pixeles (máscaras, modificación de valores, etc.).
 - Si usa códigos de terceros, debe indicar qué parte se ha tomado y de dónde (colocar citas), a fin de valorar correctamente el trabajo.
 - El proyecto debe presentarse funcionando en vivo durante la semana de revisión de exámenes.
 - **Fecha de entrega: Semana de revisión de exámenes interciclo.**

RÚBRICA DE EVALUACIÓN PARA LA PARTE I PROYECTO (15 puntos):

Proyecto Integrador

Criterio	Descripción detallada	Ponderación
1. Arquitectura de Captura y Hardware*	El sistema debe estar desarrollado en OpenCV con C++ y comunicarse exitosamente con los microcontroladores ESP32-CAM, los cuales tienen parámetros de firmware ajustados (AEC, AGC, XCLK). El código en C++ implementa hilos (threads) para la lectura concurrente de ambas cámaras, reduciendo significativamente la latencia y el jitter de red.	20% (Si usan las cámaras ESP32-CAM-MB o ESP32-XIAO-SP) 5% (Si usan webcams o similares)
2. Calidad del Mapa de Disparidad	El mapa de disparidad resultante es denso, con bordes definidos y bajo nivel de ruido (fantasmeo o huecos negros). Se evidencia la aplicación correcta y justificada de filtros de preprocesamiento (ej. CLAHE o similares) y posprocesamiento (ej. filtro WLS) vinculados al algoritmo SGBM o similares.	25%
3. Precisión y Estabilidad de la Distancia	El sistema reproyecta la matriz 3D y extrae la coordenada Z correctamente en unidades reales (cm o mm). La lectura en pantalla es estable, evidenciando la implementación matemática de un Filtro de Kalman 1D o Media Móvil para absorber las fluctuaciones lumínicas (la fluctuación no debe superar los $\pm 3 \text{ cm}$ en un entorno estático).	20%
4. Efecto de Realidad Aumentada (AR)	Se integra un efecto visual computacional original sobre la transmisión rectificada a color. El efecto reacciona dinámicamente (en escala, posición o comportamiento) a la distancia medida. Cumple con la restricción de ser un diseño exclusivo y único del grupo.	15%

5. Mini-Evaluación Individual (Defensa)	Mini-evaluación oral o escrita durante la demostración en vivo. El docente realizará preguntas puntuales sobre líneas específicas del código fuente o técnicas usadas (ej. explicación matemática de la matriz Q, funcionamiento de los bloqueos en los hilos, o sintonización de parámetros SGBM, etc.).	20% (Nota de evaluación: En trabajos en pareja, esta nota es individual. Si un estudiante no puede explicar el código, perderá la totalidad de este porcentaje independientemente del funcionamiento del programa).
Total (sobre 100%)	100%	85%

*Este criterio tiene una ponderación distinta para el tipo de implementación que se realice. Si desarrolla la opción para realizar el proyecto usando dos cámaras ESP32-CAM-MB o ESP32-XIAO-SP (como lo visto en clase), el puntaje es de 20%. Si por otra parte, en lugar de ello, se usan webcams o similares es del 5%. Esto se da con el objetivo de que los estudiantes puedan optar por aplicar una alternativa en caso de no lograr implementar la aplicación con dos cámaras ESP32.

RÚBRICA DE EVALUACIÓN PARA EL INFORME DE LA PARTE I (5 puntos):

A continuación se especifican los criterios de evaluación:

Video – Blog en inglés (Sobre 4 Puntos)		
Criterio	Ponderación	
El informe incluye una descripción detallada del trabajo realizado (introducción, descripción del problema, propuesta de solución, conclusiones, bibliografía)	35%	
El informe incorpora resultados y gráficas de pruebas realizadas con el sistema en tiempo real. Se debe evaluar específicamente:	25%	
1. Cuadros por segundo (FPS) logrados en la aplicación. 2. Análisis del error métrico en el cálculo de distancia (ej. comparativa de error midiendo objetos reales a 30, 60 y 90 cm). 3. Gráficas que demuestren la estabilización matemática obtenida al aplicar filtros (Kalman, WLS o similares) frente a la lectura cruda.		
El informe está correctamente redactado, contiene citas a papers y un esquema explicativo de la solución planteada	20%	
El vídeo en inglés sigue un guion donde se explican los principales aspectos del proyecto de una manera clara y concisa	20%	
	Total	100%
	Total	100%

b) Parte II (Final):

Debe implementar un sistema inteligente de monitoreo de tráfico organizado en dos componentes interactivos: una aplicación de escritorio desarrollada en **OpenCV C++** que detecte la presencia de un medio de transporte específico del Ecuador usando técnicas clásicas (HOG + SVM o LBP + Cascadas de Haar), y un **Bot de Telegram** (desarrollado en Python) que reciba la captura y utilice la red neuronal **YOLO** para realizar la segmentación de instancias de los objetos presentes en la escena.

Cuando la aplicación de escritorio determine que en el frame de video existe el vehículo objetivo, deberá enviar automáticamente la imagen y una secuencia corta de video al Bot de Telegram. El Bot procesará la información con YOLO, generará las máscaras de segmentación y responderá al usuario con los resultados.

1. Detector de vehículos usando técnicas clásicas (HOG + SVM o LBP + Cascadas de Haar)

- La aplicación de escritorio (OpenCV C++) debe procesar un flujo de video desde una cámara web y evaluar frame a frame si existe la presencia del vehículo asignado al grupo.
- Esta primera etapa actúa como un "disparador" (trigger) eficiente. No se requiere segmentar en C++, únicamente trazar el bounding box para determinar si el medio de transporte está en la escena.
- Cada grupo deberá enfocar su detector en un tipo de vehículo del ecosistema ecuatoriano (ej. buses alimentadores, taxis amarillos, camionetas de transporte mixto, etc.). **Ningún grupo puede tener el mismo vehículo objetivo.**

2. Construcción del Dataset y Entrenamiento

- Para lograr la detección en C++, los estudiantes deberán construir un dataset propio que contenga al menos **4.000 imágenes positivas** del vehículo objetivo (capturadas en diferentes ángulos, condiciones de luz y posturas) y al menos **4.000 imágenes negativas** (calles vacías, peatones u otros objetos que no correspondan al vehículo).
- Es necesario el uso de técnicas de aumento de datos (Data Augmentation) empleando librerías como **Albumentations** para generar robustez en el modelo (<https://albumentations.ai/>).
- El entrenamiento se realizará extrayendo los descriptores (HOG o LBP) y entrenando un clasificador SVM o similar, o mediante el uso de la utilidad `opencv_traincascade` de OpenCV (como se observa en la Ilustración 3).
 - Para ello, puede basarse en los pasos explicados en el siguiente tutorial: <https://rithikachowta.medium.com/object-detection-lbp-cascade-classifier-generation-a1d1a1c2d0b>
 - Recuerde por favor que ningún grupo puede tener el mismo dataset.**

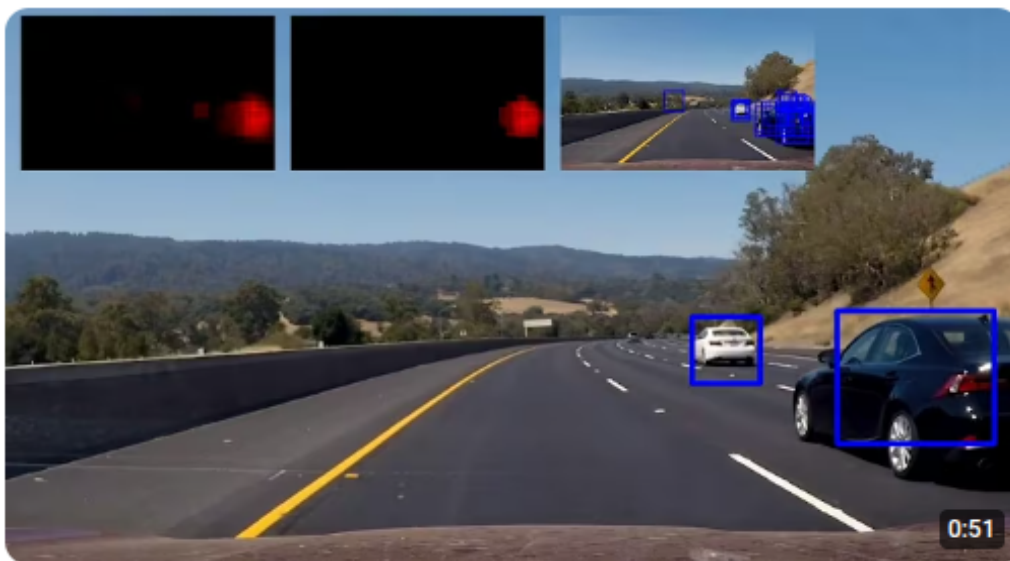


Ilustración 3: Ejemplo de una imagen donde se detecta vehículos en video usando HOG + SVM (fuente: <https://www.youtube.com/watch?v=NpjixVTNmyw>).

3. Segmentación de objetos usando Deep Learning (YOLO) en Telegram

- Una vez que la aplicación de escritorio C++ detecta el vehículo, debe enviar el frame clave y un clip de video de 5 segundos al Bot de Telegram mediante una petición a su API.
- El Bot de Telegram (desarrollado en Python) recibirá los archivos y ejecutará un modelo de **Segmentación de Instancias YOLO** (se recomienda el uso de versiones recientes y eficientes como YOLOv8-seg o YOLOv11-seg mediante la librería `ultralytics`).
- Condición de implementación:** No se requiere realizar *fine-tuning* (reentrenamiento) de la red

YOLO para la segmentación. Los estudiantes deberán instanciar el modelo preentrenado (ej. yolov8n-seg.pt) y utilizarlo en modo inferencia para segmentar e identificar a nivel de píxel todos los objetos que la red es capaz de reconocer por defecto en la escena de tráfico (vehículos, personas, semáforos, señales, etc.) (Ilustración 4).

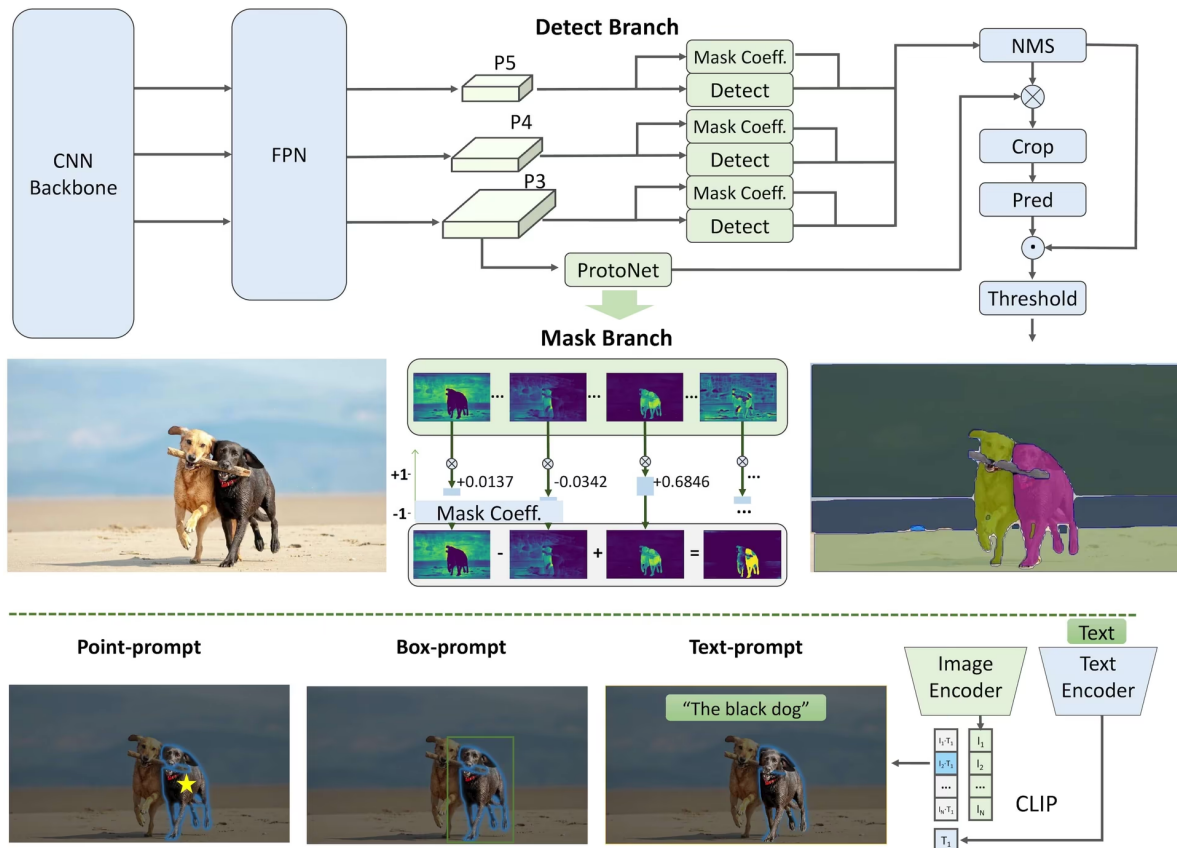


Ilustración 4: Ejemplo de segmentación usando YOLO FastSAM. Fuente: <https://docs.ultralytics.com/models/fast-sam/#model-architecture>

4. Flujo de respuesta y entregables del Bot de Telegram

El Bot debe enviar automáticamente al celular (o cuenta de Telegram) del usuario registrado los siguientes tres elementos:

- **Imagen original:** Acompañada de un mensaje de alerta indicando: "*Vehículo objetivo [Nombre del Vehículo] detectado en la escena*".
- **Imagen segmentada:** El mismo frame procesado por YOLO, donde se superpongan las máscaras de color translúcido (polígonos) delimitando el contorno exacto de cada objeto detectado y sus respectivas etiquetas.
- **Vídeo corto:** Un clip (en formato MP4 o GIF animado) de al menos 5 segundos de duración donde se observe la secuencia de video continua con la segmentación de instancias aplicada frame a frame por YOLO.

5. Aspectos generales a tomar en consideración:

- Se debe diseñar un diagrama explicativo de la arquitectura de red y el flujo de comunicación entre el programa en C++ y el Bot en Python (de forma similar a cómo se presenta en los esquemas de diseño de la Parte I).
- La aplicación de escritorio y el script del Bot de Telegram deben imprimir en consola o en pantalla información vital de rendimiento: uso de memoria RAM, cuadros por segundo (FPS) procesados en la inferencia, y el nivel de confianza (confidence) de las detecciones.
- Si se emplean repositorios o códigos de terceros para configurar la API de Telegram, debe indicar

qué parte se ha tomado y de dónde (colocar citas bibliográficas e hipervínculos) a fin de valorar correctamente la originalidad del trabajo.

- El proyecto debe presentarse funcionando en vivo durante la semana de revisión de exámenes. Los estudiantes deberán traer videos en alta calidad de entornos reales ecuatorianos para someter el sistema a pruebas de estrés frente al docente.
- **Fecha de entrega: Semana de revisión de exámenes finales.**

Documentación de Soporte:

- Tutorial detección de vehículos usando LBP y Cascadas en OpenCV:
 - <https://rithikachowta.medium.com/object-detection-lbp-cascade-classifier-generation-a1d1a1c2d0b>
- Tutorial detector de objetos usando Cascadas de Haar en OpenCV:
 - <https://machinelearningmastery.com/training-a-haar-cascade-object-detector-in-opencv/>
- Tutorial Seguimiento de Detección de la Postura Humana con OpenPose:
 - <https://blog.roboflow.com/what-is-openpose/>

RÚBRICA DE EVALUACIÓN PARA EL PROYECTO PARTE II FINAL (15 puntos):

Proyecto Integrador			
Criterio	Descripción detallada	Ponderación	
1. Detección clásica y dataset personalizado	La aplicación de escritorio (C++) detecta con precisión el vehículo ecuatoriano objetivo en el flujo de video usando HOG+SVM o LBP+Cascadas. Se evidencia la construcción de un dataset robusto (mínimo 4.000 imágenes positivas y 4.000 negativas) y la correcta aplicación de técnicas de aumento de datos (ej. Albumentations).	25%	
2. Segmentación con YOLO y respuestas del bot	El bot de Telegram (Python) procesa la alerta recibida y ejecuta exitosamente un modelo YOLO (v8 o v11 o similares) preentrenado. El bot responde automáticamente enviando los tres entregables exigidos: 1) Imagen original con alerta, 2) Imagen con máscaras de segmentación de instancias superpuestas, y 3) Clip de video de la escena segmentada.	20%	
3. Integración de software y arquitectura API*	La comunicación entre la aplicación C++ y el bot de Python se realiza de forma técnica y fluida mediante peticiones API, enviando videos e imágenes a través de flujos de datos (<i>streams</i>).	15%	5%
		(Si usa peticiones de red/API)	(Si comunican ambos programas leyendo y escribiendo archivos en una carpeta local o no envía videos)
4. Rendimiento y pruebas de estrés en vivo	El sistema muestra en pantalla métricas vitales (uso de memoria, cuadros por segundo - FPS, y nivel de confianza de la detección). El grupo supera las pruebas en vivo frente al docente, demostrando que su modelo no genera falsos positivos excesivos al evaluarlo con videos de	20%	

	tráfico ecuatoriano no vistos durante el entrenamiento.		
5. Mini-evaluación individual (Defensa)	Mini-evaluación oral durante la demostración en vivo. El docente realizará preguntas puntuales (oral o escrita) sobre la creación del dataset, la configuración del clasificador clásico, la implementación de la API de Telegram o la inferencia de YOLO.	20% (En trabajos en grupo, esta nota es individual. Si un estudiante no domina los conceptos implementados, perderá la totalidad de este porcentaje)	
Total (sobre 100%)		100%	90%

***Este criterio tiene una ponderación distinta para el tipo de implementación que se realice. Si desarrolla la opción para capturar videos y procesarlos en el Bot, el puntaje es de 15%. Si por otra parte, en lugar de ello, se captura únicamente una suencia de imágenes o se comunica escribiendo en disco la información, la ponderación es del 5%. Esto se da con el objetivo de que los estudiantes puedan optar por aplicar una alternativa en caso de no lograr capturar video.**

RÚBRICA DE EVALUACIÓN PARA EL INFORME PARTE II FINAL (5 puntos):

A continuación se especifican los criterios de evaluación:

Video – Blog en inglés (Sobre 4 Puntos)

Criterio	Ponderación
1. Estructura y Descripción del Proyecto: El informe incluye una descripción detallada del trabajo realizado. Abarca: introducción, descripción del problema (enfocado en el entorno vehicular ecuatoriano), justificación del dataset creado, propuesta de solución (arquitectura C++ y Bot), conclusiones y bibliografía.	30%
2. Análisis Cuantitativo y Cualitativo de Resultados: El informe incorpora un análisis exhaustivo que incluye: Para el detector clásico (HOG/LBP): Matriz de confusión, gráficas de precisión, sensibilidad y especificidad obtenidas al evaluar su dataset. Para el sistema integrado: Métricas de rendimiento como uso de memoria RAM (tanto en la app de escritorio como en el script de Python), cuadros por segundo (FPS) procesados, y análisis de la latencia en la comunicación con la API de Telegram. Para YOLO: Evaluación cualitativa de qué tan bien se adaptan las máscaras de segmentación generadas a los vehículos de los videos de prueba.	30%
3. Formato, Redacción y Arquitectura: El informe está correctamente redactado, contiene citas a papers y un esquema explicativo de la solución planteada de la arquitectura de comunicación implementada entre OpenCV C++ y el Bot de Telegram (Python).	20%
4. Presentación Audiovisual: El video-blog en inglés sigue un guion estructurado donde se explican los principales aspectos técnicos del proyecto, el armado del dataset y los resultados obtenidos de una manera clara y concisa.	20%
Total	100%

RESULTADO(S) OBTENIDO(S):

El desarrollo del Proyecto Integrador “**Desarrollo de Sistema Inteligente de Visión Estéreo y Detección de Vehículos Específicos en Entornos Reales**” permite a los estudiantes consolidar y aplicar de manera práctica las

	Computación	Docente: Vladimir Robles Bykbaev
	VISIÓN ARTIFICIAL	Período Lectivo: Abril – Agosto 2026

competencias definidas en los cuatro Resultados de Aprendizaje de la asignatura, demostrando una integración profunda entre teoría y aplicación.

Habilidades desarrolladas en relación con el Resultado de Aprendizaje 1: Identifica características, espacios de color y elementos de formación de imágenes.

- **Parte I (Interciclo):** Al configurar y sincronizar el arreglo de cámaras ESP32-CAM, los estudiantes identifican los elementos fundamentales de un sistema de visión artificial (sensor, modelo Pinhole implícito en la calibración estéreo) y comprenden la importancia crítica del control de firmware (exposición, ganancia, reloj) en la formación idéntica de pares de imágenes.
- **Parte II (Final):** Al capturar y curar imágenes de flujos de video de entornos viales reales, los estudiantes enfrentan los desafíos de la formación de imágenes digitales bajo condiciones de iluminación y movimiento no controladas. **Habilidad adquirida:** Los estudiantes son capaces de comprender los fenómenos involucrados en la captación de imágenes, controlar el hardware a bajo nivel para garantizar la fidelidad de la captura y utilizar matrices de calibración para transformar y rectificar geométricamente las imágenes digitales.

Habilidades desarrolladas en relación con el Resultado de Aprendizaje 2: Desarrolla algoritmos para preprocesamiento y manipulación de imágenes.

- **Parte I (Interciclo):** Implementan algoritmos de preprocesamiento avanzado como la ecualización adaptativa de histograma (CLAHE) para resaltar texturas, así como filtros matemáticos de posprocesamiento (filtro WLS) para suavizar y rellenar huecos en los mapas de disparidad crudos.
- **Parte II (Final):** Para la construcción de su dataset de vehículos, los estudiantes aplican técnicas de preprocesamiento, manipulación de píxeles y aumento de datos espaciales y de color (empleando librerías como Albumentations) para generar robustez en las muestras. **Habilidad adquirida:** Los estudiantes pueden ecualizar histogramas de forma local, implementar algoritmos de reducción de ruido estructural y manipular imágenes para estabilizar lecturas de profundidad, integrando estas técnicas en flujos de trabajo en tiempo real.

Habilidades desarrolladas en relación con el Resultado de Aprendizaje 3: Desarrolla algoritmos para la extracción de características globales y locales.

- **Parte I (Interciclo):** El uso del algoritmo StereoSGBM requiere que los estudiantes entiendan la correspondencia de bloques en la imagen, apoyándose implícitamente en el análisis de texturas y características locales a nivel de subpíxel para lograr un mapa de disparidad denso.
- **Parte II (Final):** El uso de HOG (Histogram of Oriented Gradients) o LBP (Local Binary Patterns) para la detección de vehículos ecuatorianos constituye una aplicación directa y rigurosa de la extracción de características globales y de textura sobre objetos rígidos.
- **Habilidad adquirida:** Los estudiantes emplean algoritmos como HOG/LBP para capturar matemáticamente la forma de objetos específicos, comprendiendo la importancia de la calidad y unicidad de la característica extraída para el éxito de la clasificación en entornos saturados.

Habilidades desarrolladas en relación con el Resultado de Aprendizaje 4: Desarrolla aplicaciones para extraer descriptores en el dominio del espacio y la frecuencia a fin de sustentar el reconocimiento de patrones.

- **Parte I (Interciclo):** La reproyección a 3D (mediante la matriz Q) para extraer la profundidad (Z) y la inserción interactiva de un efecto dinámico de Realidad Aumentada (AR) refuerzan la toma de decisiones algorítmicas basadas en la interpretación espacial de la escena.
- **Parte II (Final):** La integración de un clasificador en el dominio del espacio (HOG+SVM o Cascadas) junto con un modelo de aprendizaje profundo (YOLO) para la segmentación de instancias, introduce a los estudiantes en el reconocimiento de patrones moderno y su despliegue distribuido (vía API de Telegram).
- **Habilidad adquirida:** Los estudiantes comprenden los fundamentos de los algoritmos de reconocimiento de patrones clásicos e implementan sistemas de inferencia basados en redes neuronales preentrenadas (YOLO), aplicando estos conocimientos para crear soluciones de monitoreo inteligente completas y comunicativas.

CONCLUSIONES:

Al completar este Proyecto Integrador, los estudiantes de la Carrera de Computación habrán logrado una comprensión integral y aplicada de los principios fundamentales de la Visión por Computador, validando todos los Resultados de Aprendizaje de la asignatura. Las conclusiones específicas son las siguientes:

1. **Comprendieron la formación y adquisición de imágenes en contextos reales:** Los estudiantes enfrentaron los retos de la captura estereoscópica sincronizada, comprendiendo los elementos físicos y lógicos involucrados en el control a bajo nivel del hardware (sensores ESP32-CAM, parámetros de firmware, mitigación de latencia de red) y la digitalización de video. Esta habilidad les permite configurar de manera óptima los dispositivos para garantizar la fidelidad geométrica y lumínica de la información visual.
2. **Dominaron las técnicas de preprocesamiento y manipulación de imágenes:** A través de la rectificación de imágenes, la implementación de ecualización adaptativa (CLAHE) y filtros de posprocesamiento (WLS) para limpiar mapas de disparidad, así como el uso de técnicas de aumento de datos para sus datasets, los estudiantes aprendieron a evaluar el impacto de estos algoritmos para lograr mediciones estables y modelos predictivos robustos.
3. **Aplicaron con éxito la extracción de características globales y locales:** La utilización de descriptores como HOG o LBP para capturar matemáticamente la forma y textura de vehículos ecuatorianos, sumada a la comprensión del emparejamiento de bloques a nivel de subpíxel en el algoritmo StereoSGBM, les permitió entender cómo el software interpreta las características visuales frente a variaciones de iluminación y perspectiva.
4. **Implementaron sistemas de reconocimiento de patrones modernos y tradicionales:** El proyecto les permitió experimentar e integrar dos grandes paradigmas en una misma solución: el clásico (clasificadores HOG+SVM o LBP+Cascadas de Haar como disparadores eficientes) y el moderno (modelos de aprendizaje profundo preentrenados como YOLO para segmentación de instancias). Esto les proporciona una perspectiva práctica sobre cómo arquitectar sistemas híbridos que optimizan recursos computacionales.
5. **Desarrollaron habilidades transversales de ingeniería de software y comunicación:** La integración de componentes heterogéneos (programación concurrente en OpenCV C++, scripts en Python, consumo de la API de Telegram) sumada a la obligatoriedad de generar un informe analítico riguroso y un vídeo-blog en inglés, les enseñó a trabajar con arquitecturas distribuidas, gestionar el rendimiento de sus algoritmos y comunicar sus hallazgos técnicos de manera efectiva.
6. **Conectaron la teoría con la práctica en campos críticos:** Al enfocarse en la estimación precisa de profundidad interactuando con elementos de Realidad Aumentada y en el monitoreo inteligente del parque automotor local, los estudiantes apreciaron el impacto profesional de la visión artificial, asimilándola como una herramienta de ingeniería esencial para resolver problemas reales del entorno.

RECOMENDACIONES:

Para los estudiantes:

- **Planificar con anticipación las etapas del proyecto:** Dado que el proyecto está dividido en dos fases (Interciclo y Final), los estudiantes deben definir un cronograma de trabajo estricto. Asegúrese de realizar pruebas previas al día de la presentación y configurar correctamente la red en la UPS. Es altamente recomendable traer su propio router dedicado para aislar el tráfico de video de las cámaras ESP32-CAM y conectar la computadora base mediante cable de red (Ethernet) para evitar latencias, colisiones de paquetes y fluctuaciones (*jitter*) durante la evaluación.
- **Investigar y probar nuevas técnicas de visión artificial:** Además de las técnicas y filtros exigidos (CLAHE, WLS, Kalman, HOG/LBP, YOLO), es fundamental que los estudiantes investiguen y apliquen aproximaciones no contempladas en el material base (por ejemplo, el uso de filtros de partículas para estabilización, o diferentes técnicas de aumento de datos espaciales). Esta iniciativa investigativa será valorada positivamente en las rúbricas.

	Computación	Docente: Vladimir Robles Bykbaev
	VISIÓN ARTIFICIAL	Período Lectivo: Abril – Agosto 2026

Para el docente:

- **Proporcionar soporte técnico continuo:** Se recomienda ofrecer sesiones adicionales de asesoría práctica donde los estudiantes puedan resolver dudas críticas, tales como la modificación del firmware de los microcontroladores (fijación de exposición y reloj XCLK), el manejo de programación concurrente (hilos) en OpenCV C++ para la captura simultánea, y la arquitectura de peticiones a la API de Telegram.
- **Facilitar recursos de infraestructura:** Durante las fases de prueba y presentación en vivo, es imperativo coordinar con el equipo de TI para garantizar que los estudiantes tengan los permisos de red necesarios para la transmisión local de video IP y la conexión de salida para los bots de Python, evitando bloqueos por *firewalls* institucionales.

Para la Dirección de Carrera de Computación:

- **Asignar recursos para capacitación técnica:** Se sugiere que la Dirección de Carrera organice talleres previos al inicio del ciclo enfocados en ecosistemas de Internet de las Cosas (IoT) aplicados a la visión artificial (*Edge AI*), programación concurrente en C++ y el despliegue básico de modelos de *Deep Learning* preentrenados. Esto permitirá que los estudiantes lleguen mejor preparados para afrontar la fuerte integración de hardware y software que exige la asignatura.

Estas recomendaciones buscan asegurar que tanto estudiantes como docentes y la administración tengan las herramientas y el soporte necesario para completar el proyecto de manera efectiva y sin contratiempos.

Docente / Técnico Docente: Ing. Vladimir Robles Bykbaev

Firma: _____